Curso: Ciência da Computação - 3º Ano
Disciplina: Projeto e Análise de Algoritmos
Prof. Newton Spolaôr
Alunos: Thaynara Nascimento e Moisés Fayad

TRABALHO PRÁTICO DE ALGORITMOS EM GRAFOS

1. INTRODUÇÃO

O Sistema de Algoritmos em Grafos é um programa desenvolvido via interface de linha de comando que permite a modelagem, manipulação e visualização de grafos orientados e não orientados. O objetivo principal do *software* é colocar em prática a teoria dos grafos, implementando algoritmos clássicos de travessia como Busca em Largura e Profundidade, Ordenação Topológica, roteamento utilizando caminhos curtos com Dijkstra, conectividade mínima através da Árvore Geradora Mínima de Kruskal e agrupamento, com Componentes Fortemente Conexas de Kosaraju.

2. MATERIAIS E MÉTODOS

O desenvolvimento, construção e execução do sistema baseia-se na linguagem de programação Java com a versão JDK 17.0 ou superior, programado no ambiente de desenvolvimento Vs code e IntelliJ. Também se fez o uso da biblioteca externa Graphviz na versão 16.0 ou superior, que é responsável pela geração de gráficos. Este *framework* precisa estar obrigatoriamente inserido ao *PATH* do sistema operacional, viabilizando a construção de imagens *.png* a partir dos *scripts .dot* do programa.

O projeto foi manejado com uma arquitetura modular sem o uso de ferramentas externas para *build* e execução. Portanto, para compilar e executar o *software* via terminal os seguintes comandos devem ser feitos na raiz do projeto:

Compilação: Cria um diretório *bin* e compila os arquivos do pacote *src.*

```bash
javac -d bin -sourcepath src src/Main.java
```

Execução: Inicia o orquestrador do sistema.

```bash
java -cp bin Main
```

3. SISTEMA DE ALGORITMOS EM GRAFOS

As responsabilidades do sistema foram separadas visando a manutenção e legibilidade do código, conforme a Figura 1 ilustra. Os pacotes foram divididos em *assets*, src/modelo, src/algoritmos, *data*, *docs* e src/*util*. A *assets* armazena as imagens dos grafos gerados pelo Graphviz, salvando como *´meu_grafo.png´* os arquivos gerados pela opção 2 do menu principal, *´KruskalAGM.png´* a imagem gerada pelo algoritmo de Kruskal com linhas em vermelho sinalizando o caminho percorrido por todo o grafo em relação à chegada em cada vértice e *´ComponenteConexa.png´* a imagem gerada em que todos os vértices de cada componente fortemente conexa estão pintados com uma mesma cor, porém diferente da cor das demais componentes fortemente conexas, conforme exigido na especificação do trabalho.

O pacote de modelo armazena a estrutura de dados base, sendo do grafo e das arestas. Já a seção de algoritmos isola a lógica de programação de cada um dos algoritmos do sistema, sendo os clássicos de travessia, ordenação, roteamento, conectividade mínima e agrupamento. O *data* é o diretório responsável por armazenar os arquivos de entrada *.txt* do sistema e o pacote de *docs* armazena os documentos do programa, como o seminário de

apresentação do trabalho e este documento. Por fim, a pasta *util* agrupa a lógica de funcionalidade de entrada e saída e exportação para o *framework* Graphviz do programa.

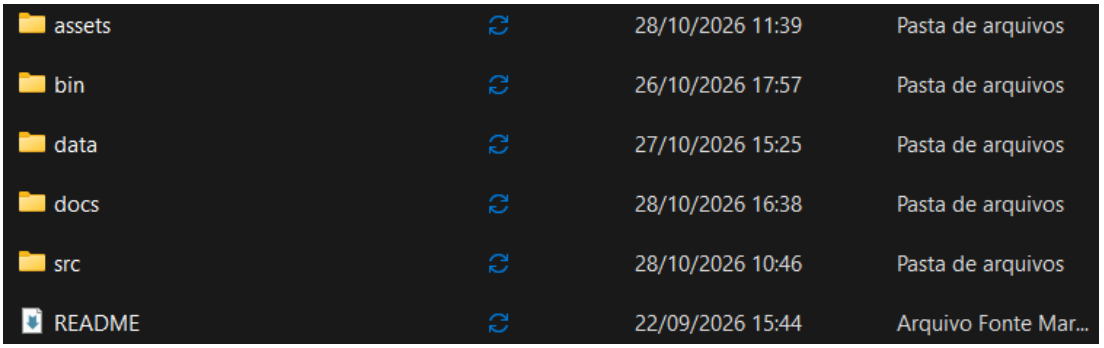


Figura 1. Arquitetura modular do sistema (Fonte: Os autores, 2026)

3.1. ESTRUTURAS DE DADOS

Para representação do grafo, optamos por utilizar Lista de Adjacências ao invés de Matrizes, para otimizar o uso de memória em grafos esparsos. A estrutura de dados base do grafo utilizou a interface List do Java, que armazena uma lista de arestas de cada vértice. A seguir, encontra-se um trecho do código-fonte.

```java
public class Grafo {
    private int numVertices;
    private boolean orientado;

    //lista que guarda uma lista de arestas para cada vértice
    private List<List<modelo.Aresta>> adjacencias;

    //construtor
    public Grafo(int numVertices, boolean orientado) {
        this.numVertices = numVertices;
        this.orientado = orientado;
        this.adjacencias = new ArrayList<>();

        //inicializa uma lista vazia para cada vértice do grafo
        for (int i = 0; i < numVertices; i++) {
            adjacencias.add(new ArrayList<>());
        }
    }

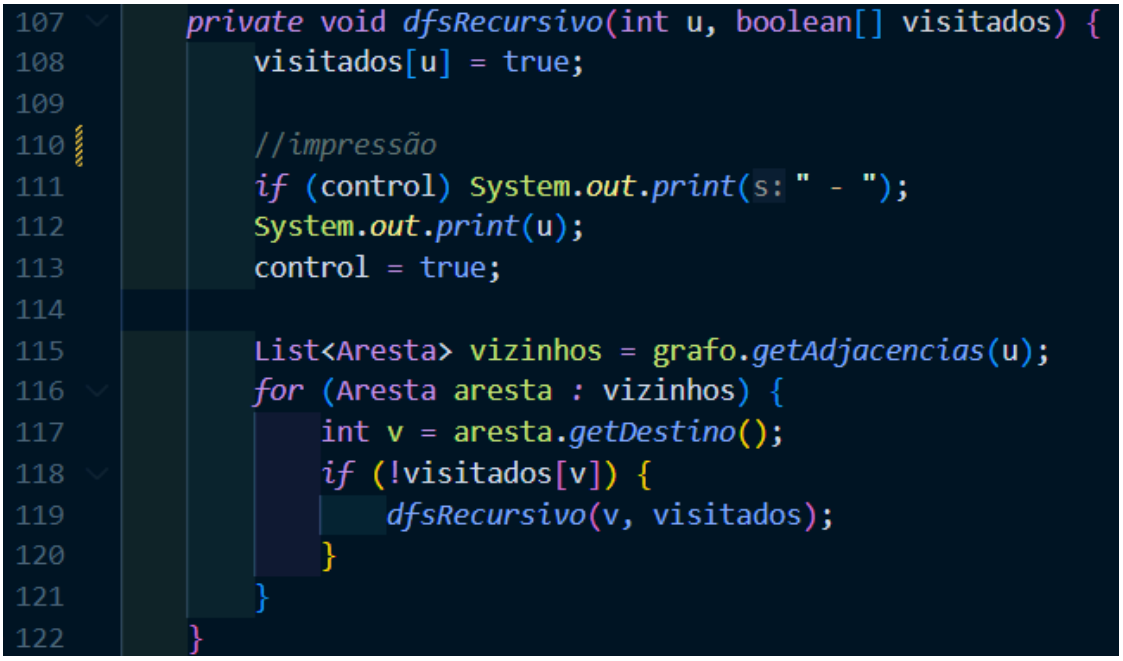
    //para popular o grafo
    public void adicionarAresta(int u, int v, int peso) {
        //cria e adiciona a aresta na lista do vértice de origem (u -> v)
        adjacencias.get(u).add(new Aresta(v, peso));
        //ordena a lista para manter a regra crescente
        Collections.sort(adjacencias.get(u));

        //se o grafo não for orientado, precisa adicionar a ida e a volta (v -> u)
        if (!orientado) {
            adjacencias.get(v).add(new Aresta(u, peso));
            Collections.sort(adjacencias.get(v));
        }
    }
```

Figura 2. Estrutura de dados base do grafo (Fonte: Os autores, 2026)

Em relação aos algoritmos de travessia, o algoritmo de Largura (BFS) foi estruturado utilizando Fila (*Queue*) do Java, permitindo a exploração em camadas. E o algoritmo de Profundidade (DFS) implementamos utilizando recursão, com o uso implícito da pilha de chamadas do sistema. Ambos possuem laços de controle que garantem a exploração de grafos mesmo que com componentes desconexas.

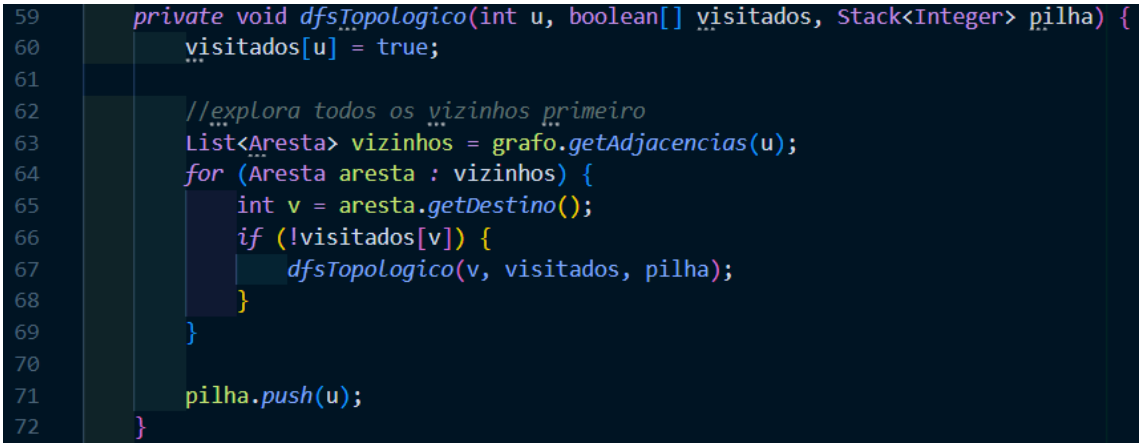
```
56      private void bfsFila(int inicio, boolean[] visitados) {
57          Queue<Integer> fila = new LinkedList<>();
58          visitados[inicio] = true;
59          fila.add(inicio);
60
61          while (!fila.isEmpty()) {
62              int u = fila.poll();
63
64              //impressão
65              if (control) System.out.print(s: " - ");
66              System.out.print(u);
67              control = true;
68
69              List<Aresta> vizinhos = grafo.getAdjacencias(u);
70              for (Aresta aresta : vizinhos) {
71                  int v = aresta.getDestino();
72                  if (!visitados[v]) {
73                      visitados[v] = true;
74                      fila.add(v);
75                  }
76              }
77          }
78      }
```



```
107     private void dfsRecursivo(int u, boolean[] visitados) {
108         visitados[u] = true;
109 
110         //impressão
111         if (control) System.out.print(s: " - ");
112         System.out.print(u);
113         control = true;
114 
115         List<Aresta> vizinhos = grafo.getAdjacencias(u);
116         for (Aresta aresta : vizinhos) {
117             int v = aresta.getDestino();
118             if (!visitados[v]) {
119                 dfsRecursivo(v, visitados);
120             }
121         }
122     }
```

Figuras 3 e 4. Estrutura de dados dos algoritmos de Busca em Largura e de Profundidade (Fonte: Os autores, 2026)

A Ordenação Topológica usa a lógica do algoritmo de DFS, diferenciando-se apenas por utilizar uma pilha (*Stack*) do Java, onde cada vértice só é empilhado após todos os seus vizinhos e dependentes terem sido completamente explorados.



```
59     private void dfsTopologico(int u, boolean[] visitados, Stack<Integer> pilha) {
60         visitados[u] = true;
61 
62         //explora todos os vizinhos primeiro
63         List<Aresta> vizinhos = grafo.getAdjacencias(u);
64         for (Aresta aresta : vizinhos) {
65             int v = aresta.getDestino();
66             if (!visitados[v]) {
67                 dfsTopologico(v, visitados, pilha);
68             }
69         }
70 
71         pilha.push(u);
72     }
```

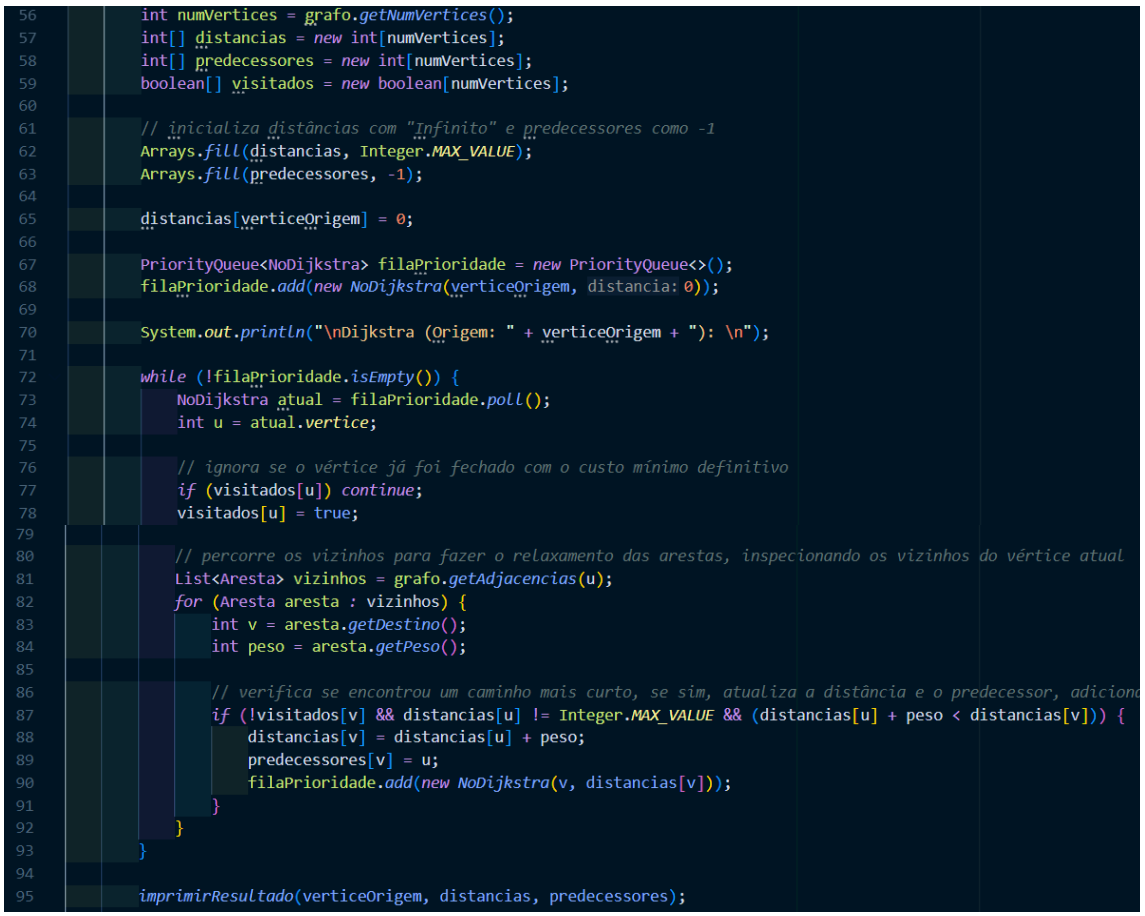
Figura 5. Trecho do código-fonte do algoritmo de Ordenação Topológica (Fonte: Os autores, 2026)

Para a Componente Fortemente Conexa foi implementado utilizando uma *Stack* de tempos de término e o grafo transposto.

```
28        int numVertices = grafo.getNumVertices();
29        Stack<Integer> pilha = new Stack<>();
30        boolean[] visitados = new boolean[numVertices];
31
32        //preenche a pilha com os vertices por ordem de termino, DFS original
33        for (int i = 0; i < numVertices; i++) {
34            if (!visitados[i]) {
35                preencherOrdem(i, visitados, pilha);
36            }
37        }
38
39        //obtem o grafo transposto
40        Grafo grafoTransposto = grafo.obterGrafoTransposto();
41
42        //faz a DFS no grafo transposto
43        visitados = new boolean[numVertices]; //reinicia o vetor de estado
44        List<List<Integer>> componentes = new ArrayList<>();
45
46        while (!pilha.isEmpty()) {
47            int v = pilha.pop();
48            if (!visitados[v]) {
49                List<Integer> componenteAtual = new ArrayList<>();
50                dfsTransposto(grafoTransposto, v, visitados, componenteAtual);
51
52                //ordena os vertices numericamente dentro da componente
53                Collections.sort(componenteAtual);
54                componentes.add(componenteAtual);
55            }
56        }
```

Figura 6. Lógica do algoritmo de Componente Fortemente Conexa (Fonte: Os autores, 2026)

Para garantir a eficiência ao buscar o caminho de menor custo no algoritmo de Dijkstra, utilizamos uma Fila de Prioridade (*PriorityQueue - Min-Heap*), que entrega sempre o vértice alcançável mais barato a cada iteração, realizando o relaxamento das arestas.



```java
int numVertices = grafo.getNumVertices();
int[] distancias = new int[numVertices];
int[] predecessores = new int[numVertices];
boolean[] visitados = new boolean[numVertices];

// inicializa distâncias com "Infinito" e predecessores como -1
Arrays.fill(distancias, Integer.MAX_VALUE);
Arrays.fill(predecessores, -1);

distancias[verticeOrigem] = 0;

PriorityQueue<NoDijkstra> filaPrioridade = new PriorityQueue<>();
filaPrioridade.add(new NoDijkstra(verticeOrigem, distancia: 0));

System.out.println("\nDijkstra (Origem: " + verticeOrigem + "): \n");

while (!filaPrioridade.isEmpty()) {
    NoDijkstra atual = filaPrioridade.poll();
    int u = atual.vertice;

    // ignora se o vértice já foi fechado com o custo mínimo definitivo
    if (visitados[u]) continue;
    visitados[u] = true;

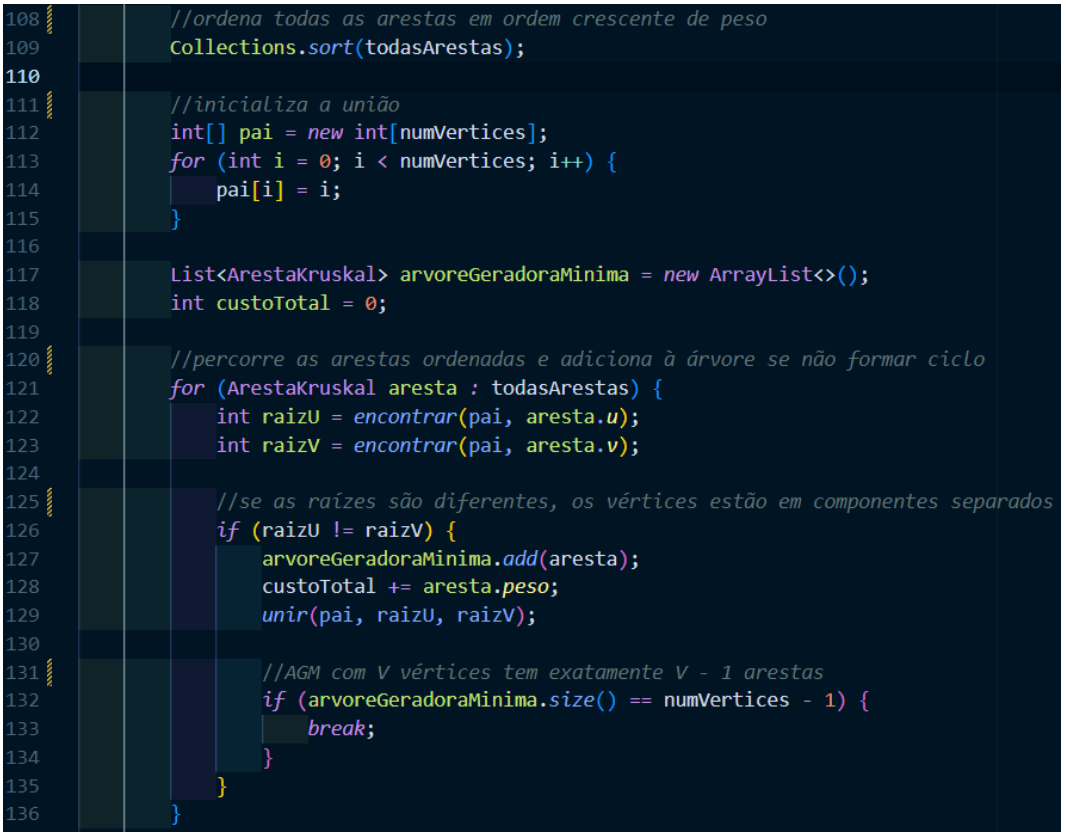
    // percorre os vizinhos para fazer o relaxamento das arestas, inspecionando os vizinhos do vértice atual
    List<Aresta> vizinhos = grafo.getAdjacencias(u);
    for (Aresta aresta : vizinhos) {
        int v = aresta.getDestino();
        int peso = aresta.getPeso();

        // verifica se encontrou um caminho mais curto, se sim, atualiza a distância e o predecessor, adicion
        if (!visitados[v] && distancias[u] != Integer.MAX_VALUE && (distancias[u] + peso < distancias[v])) {
            distancias[v] = distancias[u] + peso;
            predecessores[v] = u;
            filaPrioridade.add(new NoDijkstra(v, distancias[v]));
        }
    }
}

imprimirResultado(verticeOrigem, distancias, predecessores);
```

Figura 7. Trecho do algoritmo de Dijkstra (Fonte: Os autores, 2026)

Por fim, o método guloso de Kruskal exige a ordenação global de todas as arestas pelo menor peso. Para a detecção rápida de ciclos na montagem da árvore, implementamos a estrutura de Conjuntos Disjuntos (*Union-Find*) com compressão de caminho.



```java
//ordena todas as arestas em ordem crescente de peso
Collections.sort(todasArestas);

//inicializa a união
int[] pai = new int[numVertices];
for (int i = 0; i < numVertices; i++) {
    pai[i] = i;
}

List<ArestaKruskal> arvoreGeradoraMinima = new ArrayList<>();
int custoTotal = 0;

//percorre as arestas ordenadas e adiciona à árvore se não formar ciclo
for (ArestaKruskal aresta : todasArestas) {
    int raizU = encontrar(pai, aresta.u);
    int raizV = encontrar(pai, aresta.v);

    //se as raízes são diferentes, os vértices estão em componentes separados
    if (raizU != raizV) {
        arvoreGeradoraMinima.add(aresta);
        custoTotal += aresta.peso;
        unir(pai, raizU, raizV);

        //AGM com V vértices tem exatamente V - 1 arestas
        if (arvoreGeradoraMinima.size() == numVertices - 1) {
            break;
        }
    }
}
```

Figura 8. Trecho de código do algoritmo de Kruskal (Fonte: Os autores, 2026)

4. COMO UTILIZAR O SISTEMA

Ao executar o programa, o menu principal é exibido no console. O sistema requer o upload de um arquivo *.txt* do grafo contendo uma *flag* de orientação na primeira linha (orientado=sim ou orientado=nao), a quantidade de vértices na segunda (*V=n*), e as arestas nas linhas subsequentes no formato (*origem, destino):peso.*

4.1. OPÇÕES DO MENU

1. Carregar grafo: Deve ser obrigatoriamente a primeira ação. O usuário insere o caminho do arquivo (ex: *data/grafo_teste.txt*) para instanciar o grafo na memória.
2. Desenho de grafo: Invoca o Graphviz e gera um arquivo *.png* com o *layout* padrão do grafo inserido, ilustrando todos os vértices, arestas e seus pesos.
3. Busca em Profundidade: Imprime a ordem de visitação dos vértices explorando os ramos até sua profundidade máxima. Alcança todos os vértices, mesmo os desconexos.
4. Busca em Largura: Solicita um vértice de origem e imprime a ordem de visitação em formato de camadas.
5. Ordenação Topológica: Aplicável apenas a grafos orientados. Imprime a ordem linear de precedência dos vértices.
6. Componentes Fortemente Conexas: Aplicável apenas a grafos orientados. Imprime as componentes no console por ordem decrescente de tamanho e gera um gráfico *.png* colorindo cada componente com uma cor distinta.
7. Dijkstra: Solicita um vértice de origem e lista o custo mínimo e a rota exata para alcançar todos os outros vértices do grafo.
8. Kruskal: Lista as arestas que compõem a Árvore Geradora Mínima, exibe o peso total e gera um gráfico *. png* onde as arestas pertencentes à árvore são destacadas na cor vermelha.
0. Sair: Encerra a aplicação.

5. CONCLUSÃO

O desenvolvimento do nosso *software* demonstrou na prática a viabilidade e a eficiência das estruturas de dados mostradas em sala de aula pelo professor Newton. A migração da lógica algorítmica teórica para o trabalho prático que modularizamos em Java superou desafios relacionados ao gerenciamento de estado, como grafos desconexos, e à otimização de complexidade de tempo por meio de estruturas como *Min-Heaps* e *Union-Find*. Além disso, a integração com o *framework* Graphviz agregou um enorme valor didático, permitindo a validação visual e interativa dos resultados obtidos no console.

6. DECLARAÇÃO DE USO DE IA GENERATIVA

O Gemini (Google) foi utilizado como ferramenta de IA generativa para auxiliar na organização da arquitetura, com sugestões para maior otimização da interface do sistema e verificação dos cenários possíveis em prol da segurança e controle do código-fonte. Todo o conteúdo gerado foi avaliado de forma crítica e revisado quando necessário, permanecendo os autores integralmente responsáveis pela precisão e pela integridade do trabalho final.

7. REFERÊNCIAS BIBLIOGRÁFICAS

[1]. ZIVIANI, N. **Projeto de algoritmos**: com implementações em Java e C++. São Paulo: Cengage Learning, 2007.

[2]. CORMEN, T., LEISERSON, C. and RIVEST, R. **Introduction to Algorithms**. McGrawHill, 1997.

[3]. DIESTEL, R. **Graph Theory**. Springer, 1997.

[4]. P.O. NETTO B. **Grafos**: Teoria, Modelos, Algoritmos. Blucher, 1996.

[5]. BONDY, J. and MURTY U. S. R. **Graph Theory with Applications**. Elsevier, 1982.

[6]. DRICHEL, A. **File:depth-first-tree.svg**. Disponível em: https://commons.wikimedia.org/wiki/File:Depth-first-tree.svg, 2008. Acesso em: out., 2026.

[7]. HALIM, S. and HALIM, F. **VisuAlgo**: Graph transversal (DFS/BFS). Disponível em: https://visualgo.net/en/dfsbfs, 2026. Acesso em: out., 2026.

[8]. HUEI, D. L. **Algoritmos em grafos**, 2006. Notas didáticas.

[9]. ORLIN, J. and MONARD, M. C. **Busca em profundidade**, 2006. Notas didáticas.

[10]. ROSA, J. L. G. **Grafos**, [2010?]. Notas didáticas.

[11]. B. Matheny. **File:animated bfs.gif**. Disponível em: https://upload.wikimedia.org/wikipedia/commons/4/46/AnimatedBFS.gif, 2007. Acesso em: out., 2026.

[12]. MARTINS A. P. and HUEI D. L. **Ordenação topológica**, 2006. Notas didáticas.

[13]. KULKARNY. M. **Decision trees for classification**: A machine learning algorithm. Disponível em: https://www.xoriant.com/blog/decision-trees-for-classification-a-machine-learning-algorithm, 2017. Acesso: out., 2026.

[14]. MARTINS A. P., HUEI D. L., MONARD, M. C., and BUENO, J. M. **Grafos**, 2006. Notas didáticas.

[15]. MARTINS A. P. and HUEI D. L. **Árvore geradora mínima (mst)**, 2006. Notas didáticas.

[16]. KALININ, A. **File:shortest path with direct weights.svg**. Disponível em: https://en.wikipedia.org/wiki/File:Shortest. path with direct weights.svg, 2013. Acesso em: out., 2026.

[17]. WIKIPEDIA. **Minimum spanning tree**. Disponível em: https://en.wikipedia.org/wiki/Minimum, 2026. Acesso em: out., 2026.

[18]. PIMENTEL, M. and CARVALHO, F. **IA generativa e educação**: práticas e teorizações. Sociedade Brasileira de Computação, 2025. Disponível em: https://books-sol.sbc.org.br/index.php/sbc/catalog/book/182. Acesso em: out., 2026.

[19]. ORGANIZAÇÃO DAS NAÇÕES UNIDAS PARA A EDUCAÇÃO, A CIÊNCIA E A CULTURA. **Guia para a IA generativa na educação e na pesquisa**. Organização das Nações Unidas para a Educação, a Ciência e a Cultura, 2024. Disponível em:

https://www.unesco.org/pt/articles/guia-para-ia-generativa-na-educacao-e-na-pesquisa. Acesso em: out., 2026.

[20]. ELLSON, J. et al. **Graphviz:** Open Source Graph Drawing Tools. Disponível em: http://www.graphviz.org. Acesso em: out. 2026.

[21]. ELSEVIER. Generative AI policies for journals. Elsevier, Disponível em: https://www.elsevier.com/about/policies-and-standards/generative-ai-policies-for-journals, 2025. Acesso em: out., 2026.

[22]. UNIVERSIDADE ESTADUAL PAULISTA. Guia para a utilização de IA na graduação da Unesp: integridade, inovação equidade. Universidade Estadual Paulista; Disponível em: https://www2.unesp.br/Home/laf/guia-graduacao.pdf, Acesso em: abr. 2026, 2026.